# Vizualizer – Web Dashboard

The `vizualizer/` folder contains a Next.js web application that replaces and extends the legacy Tkinter UI. It provides scenario management, run orchestration, and results browsing through a browser at `http://localhost:3000`.

> [!NOTE]
> The intentional spelling is **vizualizer** (with a z), matching the folder name.

---

## Stack

| Layer | Technology |
|---|---|
| Framework | Next.js 16.2.4 (App Router) |
| UI | React 19.2.4, Tailwind CSS 4 |
| Language | TypeScript 5 |
| CSV parsing | papaparse 5 |
| YAML parsing | yaml 2 |
| Runtime | Node.js (server-side API routes) |

---

## Starting the Dashboard

```
cd vizualizer
npm install       # only needed once
npm run dev       # starts dev server at http://localhost:3000
```

Production build:

```
npm run build
npm start
```

---

## Pages

### Scenario Builder

- **Run mode selector:** `Paired (baseline + scenario)` or `Single scenario with reference baseline`

- **Form controls** for all scenario parameters: countries, snapshot window, cutout year, clusters, solver, stress-test shocks
- **YAML editor** — shows generated working YAML, editable before enqueue
- Template is read-only (`personal_docs/scenario_template.yaml`); the editor edits a working copy only

### Runs

- Enqueue jobs into a sequential queue (one active at a time)
- Per-job status: `queued` → `running` → `succeeded` / `failed` / `cancelled` / `interrupted`
- Live progress message (last stdout/stderr line)
- Log file viewer (last 200 lines from `logs/planui-web/<jobId>.log`)
- Cancel button (immediate for queued; kills process for running)
- On restart, any job that was `running` is marked `interrupted`

### Results

- Auto-scans `results/` for comparison output folders
- Only shows folders that contain all 7 required CSVs (see below)
- Tabs per result:
  - **Summary** – parsed cost delta, ENS, imports, price stats
  - **CSV Data** – dropdown + table preview for each required CSV
  - **Figures** – PNG viewer with file picker
  - **Assumptions** – renders `assumptions_limitations.md`

---

## API Endpoints

All endpoints are Next.js Route Handlers under `src/app/api/`.

| Method | Path | Purpose |
|---|---|---|
| `GET` | `/api/scenario/template` | Load canonical YAML template |
| `POST` | `/api/scenario/build` | Build working YAML from form inputs |
| `POST` | `/api/runs/enqueue` | Generate configs + enqueue job |
| `GET` | `/api/runs/jobs` | List all job records |
| `POST` | `/api/runs/cancel` | Cancel a job by ID |
| `GET` | `/api/runs/log` | Tail job log (`?jobId=…`) |
| `GET` | `/api/runs/baselines` | List solved baseline network files |
| `GET` | `/api/results` | Scan and list valid result entries |
| `GET` | `/api/results/summary` | Parsed summary + file list (`?name=…`) |
| `GET` | `/api/results/csv` | CSV preview (`?name=…&file=…`) |

| Method | Path | Purpose |
|---|---|---|
| GET | /api/results/figure | Serve figure PNG (?name=…&file=…) |
| GET | /api/results/assumptions | Assumptions markdown (?name=…) |

## File Paths Used

All paths are resolved relative to the repo root (one level up from vizualizer/).

| Purpose | Path |
|---|---|
| Results output | results/ |
| Generated configs | config/adversarial/generated/ |
| Scenario templates | personal_docs/ |
| Job logs | logs/planui-web/<jobId>.log |
| State persistence | vizualizer/.data/planui-state.json |

## Runtime Detection (Conda Auto-Discovery)

On startup, src/app/lib/runtime.ts auto-detects how to run Snakemake and Python. Priority order (first match wins):

1. PLANUI_CONDA_ENV env var — use named env directly (conda run -n <name>) — highest priority
2. PLANUI_CONDA_PREFIX env var — use prefix path directly (conda run -p <prefix>)
3. CONDA_PREFIX env var — active conda prefix, **only if** CONDA_DEFAULT_ENV **is not** base (guards against using the base environment when the dev server is launched without activating the project env)
4. CONDA_DEFAULT_ENV — currently active named env (skipped if base)
5. **Candidates:** pypsa, pypsa-eur — first one found in conda env list wins
6. **Fallback:** plain python / python -m snakemake (system Python)

> [!WARNING]
> If you start npm run dev from a terminal where the **base** conda environment is active, CONDA_PREFIX points to the Anaconda root (no Snakemake). The base-env guard (step 3) catches this and falls through to the named-env candidates. Still, the safest practice is to activate the pypsa environment before starting the server, or set PLANUI_CONDA_ENV explicitly.

To override, set the env var before starting the dev server:

```
$env:PLANUI_CONDA_ENV = "pypsa"    # PowerShell
# or
set PLANUI_CONDA_ENV=pypsa         # CMD
```

### Snakemake Remote Check Flag

By default, `--storage-cached-http-skip-remote-checks` is passed to every Snakemake invocation. To disable:

```
$env:PLANUI_SNAKEMAKE_SKIP_REMOTE_CHECKS = "0"
```

### Proxy Stripping

HTTP proxy environment variables are stripped before spawning subprocesses to prevent conda/snakemake from routing traffic through corporate proxies unintentionally. To keep them:

```
$env:PLANUI_USE_SYSTEM_PROXY = "1"
```

---

## Job Lifecycle

```
POST /api/runs/enqueue
    ↓ buildConfigs() → writes baseline + scenario YAMLs to
config/adversarial/generated/
    ↓ buildCommands() → constructs argv arrays for snakemake unlock, solve,
report
    ↓ jobRunner.enqueue(spec) → writes to planui-state.json
    ↓ runNext() → if no active job, starts immediately

Per-command sequence (paired mode):
  1. snakemake --unlock --configfile <baseline_cfg>     [allowFailure]
  2. snakemake -c all <baseline_target_nc> --configfile <baseline_cfg>
  3. snakemake --unlock --configfile <scenario_cfg>     [allowFailure]
  4. snakemake -c all <scenario_target_nc> --configfile <scenario_cfg>
  5. python scripts/report_romania_winter_stress.py \
       --baseline-net <baseline_nc> --scenario-net <scenario_nc> \
       --country <country> --outdir results/<output_name>
```

---

## Required Result CSVs

A result folder is only displayed if all seven files are present:

1. `system_cost_comparison.csv`
2. `generation_mix_mwh.csv`
3. `lmp_summary_ro.csv`
4. `ens_summary.csv`
5. `curtailment_mwh.csv`

6. `daily_net_imports_mwh.csv`
7. `interconnector_flow_congestion.csv`

---

## Known Environment Issue (Python 3.13 + linopy)

**Symptom:** `ModuleNotFoundError: No module named 'pkg_resources'` when Snakemake starts.

**Root cause:** `linopy` imports `google-cloud-storage` at module load time. The old bundled version `1.31.2` uses the deprecated `pkg_resources` API which is broken in Python 3.13.

**Fix:**

```
conda run -n pypsa pip install "google-cloud-storage>=2.10"
```

Modern `google-cloud-storage` (2.x) uses `importlib.metadata` instead of `pkg_resources` and resolves the import chain.

---

## Relationship to Legacy Tkinter UI

| Feature | Tkinter UI (`personal_dashboard/`) | Web UI (`vizualizer/`) |
|---|---|---|
| Scenario builder | Yes | Yes |
| Run queue | Yes | Yes |
| Results viewer | Yes | Yes |
| Bilingual EN/RO | Yes | Planned |
| Browser access | No | Yes |
| Remote use | No | Yes (any device on LAN) |
| Status | Legacy / stable | Active development |

The Tkinter UI remains functional. The web dashboard is the intended long-term replacement.